Bow & Tie — Backend Developer Handoff

Project: Bow & Tie (a.k.a. BowClips) — boutique e-commerce store for handcrafted hair bows, clips & accessories **Market:** Dhaka, Bangladesh · Currency: **BDT (৳) Prepared:** 9 July 2026 **Audience:** Incoming backend developer — you will finish the backend, test the full project end-to-end, and integrate the live third-party services (WhatsApp Business, SSLCommerz payment gateway, SMTP email, couriers, social login).

---

1. What this project is

A complete, working full-stack e-commerce application:

- **Storefront (customer-facing)** — browse catalog, search/filter, product galleries, wishlist, cart, checkout, guest checkout, order history + tracking, reviews with photos, product Q&A, newsletter.
- **Admin panel** (`/admin`) — dashboard, products (+ bulk import from CSV/Excel/PDF), orders (+ printable invoices & courier shipping), customers, promotions, coupons, sales reports, product Q&A moderation, staff accounts with per-section permissions, settings.
- **API + database** — Node + Express + Prisma. All money math (prices, discounts, shipping, totals) is **recomputed server-side** — the client is never trusted.

The app **runs today** with zero external accounts. Every third-party integration (email, WhatsApp, couriers, social login, payment) uses a **dev-fallback**: when credentials are absent it logs to the console / returns a mock, and goes fully live the moment you add the credentials. **Your job is largely to supply those credentials and flip these from mock to live — the wiring already exists.**

> **Using an AI coding assistant?** The repo root has an `AGENTS.md` that Cursor, Claude Code, GitHub Copilot, Windsurf, etc. auto-load — it gives the AI the architecture, commands, and hard rules with no setup. If you paste project context into a chat AI, share the **Markdown** version of this document (`Backend-Developer-Handoff.md`), not the PDF — plain text parses far more reliably.

Tech stack

Layer	Technology
Backend runtime	Node.js (24+), TypeScript run via `tsx` (no build step in dev)
Web framework	Express 4
ORM / DB	Prisma 6 — **SQLite** locally (`backend/prisma/dev.db`), **PostgreSQL** for production
Validation	Zod
Auth	JWT (`jsonwebtoken`) + `bcryptjs` password hashing
Uploads	Multer (product & review images → `backend/uploads/`)
Email	Nodemailer (SMTP)
Social login	`google-auth-library` + Facebook Graph API
Bulk import	`xlsx` (SheetJS) + `pdf-parse`
Frontend	React 19 + Vite + TypeScript, react-router-dom, Recharts (lazy)
State	React Context (StoreContext / AuthContext / ProductsContext)

---

2. Repository layout

```
Bow and Tie/
├── backend/                # Node + Express + Prisma API
│   ├── prisma/
│   │   ├── schema.prisma     # DB models (SQLite dev, Postgres prod)
│   │   ├── seed.ts           # demo data loader
│   │   ├── seed-data.ts      # product/promo catalog
│   │   └── dev.db            # local SQLite file (git-ignored)
│   ├── src/
│   │   ├── index.ts          # entrypoint: connect DB, start server + schedulers
│   │   ├── app.ts            # Express app + route mounting + CORS
│   │   ├── config.ts         # ALL env vars + business rules (shipping zones etc.)
│   │   ├── prisma.ts         # Prisma client singleton
│   │   ├── middleware/
│   │   │   ├── auth.ts        # requireAuth, requireAdmin, requireStaff, checkPermission
│   │   │   └── error.ts       # notFound + central error handler
│   │   ├── lib/
│   │   │   ├── mailer.ts      # SMTP send (dev-fallback → console)
│   │   │   ├── emails.ts      # HTML email templates
│   │   │   ├── whatsapp.ts    # WhatsApp Cloud API alerts + weekly/monthly scheduler
│   │   │   ├── courier.ts     # Pathao / Steadfast / RedX consignments
│   │   │   ├── oauth.ts       # Google / Facebook token verification
│   │   │   ├── promotions.ts  # active-promotion pricing
│   │   │   ├── importProducts.ts # CSV/Excel/PDF parsing for bulk import
│   │   │   ├── abandonedCart.ts  # reminder sweeper
│   │   │   ├── stockAlerts.ts     # back-in-stock notifications
│   │   │   ├── serialize.ts   # DB row → API JSON (hides costPrice etc.)
│   │   │   ├── user.ts        # user serializer (adds resolved permissions)
│   │   │   ├── auth.ts        # hash/verify password, sign/verify JWT
│   │   │   └── ids.ts        # order-id generator
│   │   └── routes/
│   │       ├── auth.ts        # register, login, oauth, forgot/reset, me
│   │       ├── products.ts    # catalog, reviews, questions, notify-me
│   │       ├── orders.ts      # create, mine, track, cancel, return
│   │       ├── account.ts     # profile, addresses, change password
│   │       ├── cart.ts        # abandoned-cart tracking
│   │       ├── newsletter.ts  # subscribe
│   │       ├── promotions.ts  # public active promos
│   │       ├── upload.ts      # admin + customer image upload
│   │       └── admin.ts       # everything under /api/admin (see §6)
│   ├── uploads/               # uploaded images (git-ignored)
│   ├── .env.example           # copy to .env and fill in
│   └── package.json
├── frontend/                # Vite + React storefront + admin
│   ├── src/
│   │   ├── pages/             # storefront pages
│   │   ├── admin/pages/       # admin panel pages
│   │   ├── components/        # shared UI
│   │   ├── services/          # db.ts (API client), admin.ts, api.ts (fetch+token)
│   │   └── store/             # Context providers
│   └── .env.example
├── proposal/                # this document + client proposal/invoice
├── docker-compose.yml         # optional local PostgreSQL
└── README.md
```

3. Running it locally (first 10 minutes)

Two terminals. Node 20+ recommended (developed on Node 24).

Backend

```
cd backend
npm install
cp .env.example .env          # then edit if needed (works as-is for dev)
npm run prisma:generate       # generate Prisma client
npm run prisma:push           # create the SQLite schema
npm run seed                  # load demo products, promos, accounts
npm run dev                   # API → http://localhost:4000
```

Frontend

```
cd frontend
npm install
cp .env.example .env
npm run dev                   # storefront → http://localhost:5173
```

Health check: `GET http://localhost:4000/api/health` → `{ "ok": true }`

Demo accounts (from the seed)

Role	Email	Password
Admin	`admin@bowclips.com`	`admin123`
Customer	`demo@bowclips.com`	`demo123`

Admin panel: log in as admin, then go to `http://localhost:5173/admin`.

> **Windows note:** Prisma's query engine DLL gets locked while the dev server runs. If `prisma:push` / `prisma:generate` fails with `EPERM`, stop the Node process first (`taskkill /F /IM node.exe`), run the Prisma command, then restart `npm run dev`.

---

4. Data model (Prisma) — quick tour

Full schema: `backend/prisma/schema.prisma`. Key conventions:

- **Money is stored as integers in BDT** (whole taka). No floats for currency.
- SQLite has no array/JSON column type, so **arrays are stored as JSON strings** (`colors` , `sizes` , `gallery` , `timeline` , `permissions` , review `images` , etc.) and parsed by the serializers. When you migrate to Postgres these still work as `String` ; you may optionally switch them to `Json` / `String[]` later.
- `costPrice` on Product is the buying cost — used for profit/margin reports and **never exposed** by the public serializer.

Models: `User` (role: customer/admin/staff + `permissions`), `Address` , `Product` , `Review` (with photo `images`), `Question` (product Q&A), `Order` + `OrderItem` , `Promo` (coupon codes), `Promotion` (campaigns/banners), `NewsletterSubscriber` , `AbandonedCart` , `StockAlert` .

`Order` carries: `deliveryZone` (inside/outside Dhaka), `payment` (cod/bkash/nagad), `txnId` , `courier` + `trackingCode` , `status` , and a JSON `timeline` .

---

5. Auth & permissions (important for testing)

- **JWT** in the `Authorization: Bearer <token>` header. Issued on register / login / oauth.
- `requireAuth` — any logged-in user.

- `requireAdmin` — **reads the role from the database** (not just the token claim), so it's safe against stale tokens.
- `requireStaff` — allows `admin` OR `staff`; attaches the user's `permissions` array to the request.
- `checkPermission` — maps the request path to a section key (`PERM_MAP` in `middleware/auth.ts`) and enforces it. **Admins bypass; staff are limited to their granted sections.** Enforcement is server-side, not just hidden in the UI.

Admin sections: `dashboard, products, import, orders, customers, promotions, coupons, reports, questions, settings, staff`. Staff can be granted any subset except `staff` (only admins manage staff).

---

6. API surface (all mounted under `/api`)

Public / auth — `routes/auth.ts`

Method	Path	Notes
POST	`/auth/register`	name, email, password → `{token, user}`
POST	`/auth/login`	email, password
POST	`/auth/oauth`	`{provider: 'google'|'facebook', token}` — verifies, find-or-creates
POST	`/auth/forgot-password`	emails a reset link (dev: console)
POST	`/auth/reset-password`	id, token, password
GET	`/auth/me`	current user

Products & content — `routes/products.ts`

Method	Path	Notes
GET	`/products`	catalog (public serializer, no costPrice)
GET	`/products/:id`	one product
GET	`/products/:id/reviews`	list
POST	`/products/:id/reviews`	auth — rating, title, text, up to 4 photo URLs
GET	`/products/:id/questions`	public Q&A
POST	`/products/:id/questions`	auth — ask a question
POST	`/products/:id/notify-me`	back-in-stock email signup

Orders — `routes/orders.ts`

Method	Path	Notes
POST	`/orders`	create — **server recomputes** subtotal/discount/shipping/total, decrements stock, fires order email + WhatsApp alert
GET	`/orders/mine`	auth — order history
GET	`/orders/:id`	auth — order detail
GET	`/orders/track/:id`	public tracking

Method	Path	Notes
POST	`/orders/:id/cancel`	auth
POST	`/orders/:id/return`	auth — request return

Account — `routes/account.ts`: `PUT /account/profile`, `POST /account/addresses`, `DELETE /account/addresses/:id`, `PUT /account/password`.

Other public: `POST /cart/track` (abandoned cart), `POST /newsletter/subscribe`, `GET /promotions/active`, `GET /promotions/:id`, `POST /upload` (auth — review photos), `POST /admin/upload` (admin — product images).

Admin — `routes/admin.ts` (all behind `requireAuth → requireStaff → checkPermission`) `GET /admin/stats` · orders: `GET /orders`, `GET /orders/:id`, `POST /orders/:id/ship`, `PATCH /orders/:id/status` · products: `GET/POST /products`, `PUT/DELETE /products/:id`, `POST /products/import`, `POST /products/import/commit` · `GET/PUT/DELETE /questions[/:id]` · WhatsApp: `POST /whatsapp/test`, `GET /whatsapp/report` · `GET /reports` (sales/profit) · `GET /customers` · promotions & coupons CRUD · staff: `GET/POST /staff`, `PUT/DELETE /staff/:id`.

---

7. Third-party integrations — status & what you need to do

Every integration below already has working code and a dev-fallback. **To go live: create the account, get credentials, put them in `backend/.env` (and `frontend/.env` where noted), and — for couriers/payment — replace the remaining mock/TODO with the real API call.**

7.1 SSLCommerz payment gateway — ⚠️ NOT YET INTEGRATED (your main build task)

Current state: Checkout accepts `payment: 'cod' | 'bkash' | 'nagad'` with an optional manual `txnId`. There is **no online payment gateway yet** — this is the primary integration to build.

What to build (SSLCommerz hosted checkout / EasyCheckout):

1. Add credentials to `config.ts` + `.env`: `SSLC_STORE_ID`, `SSLC_STORE_PASSWORD`, `SSLC_IS_LIVE` (sandbox vs live).
2. On order creation for an online payment, call SSLCommerz **Session API** (`/gwprocess/v4/api.php`) with the server-computed `total`, order id (`tran_id`), and success/fail/cancel/IPN callback URLs. Redirect the shopper to the returned `GatewayPageURL`.
3. Add callback routes: `POST /api/payments/sslcommerz/success|fail|cancel` and an **IPN** handler. On success, **validate the transaction server-side** via the Validation API (`validationserverAPI` using `val_id`) and confirm the `amount` + `tran_id` match the order before marking it paid — never trust the redirect alone.
4. Extend the `Order` model with payment status fields (e.g. `paymentStatus`, `sslcVaId` / `val_id`, gateway response). Only set status to `Confirmed`/paid after IPN validation.
5. Keep COD working as-is (no gateway round-trip).

Where to hook in: order creation is in `routes/orders.ts` (`POST /orders`, ~line 81+). The money is already computed there; branch on `payment` to either finish as COD or start an SSLCommerz session. Frontend checkout is `frontend/src/pages/CheckoutPage.tsx`.

Test cards: use SSLCommerz **sandbox** first (`developer.sslcommerz.com`). Charges/fees are the merchant's (typically ~1.5–2.5% per transaction) — the store owner registers the merchant account.

7.2 WhatsApp Business alerts — ✅ wired, needs credentials

Current state: Fully implemented via **Meta WhatsApp Cloud API** in `lib/whatsapp.ts`. Sends: new-order alert, low-stock alert, restock alert, and **weekly (Mon 9am) + monthly (1st, 9am)** sales reports (scheduler in `index.ts`). Without credentials it logs every message to the console.

To go live:

1. Create a Meta app + WhatsApp Business account, add a sender phone number, get a permanent access token.

2. Set `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`, `WHATSAPP_ADMIN_TO` (admin number, international format, no `+`), optional `LOW_STOCK_THRESHOLD` (default 5).
3. Test via `POST /api/admin/whatsapp/test`. On-demand report: `GET /api/admin/whatsapp/report?period=weekly|monthly`.

> Note: these are **admin notifications** to the shop owner. If the client later wants to send order updates **to customers** on WhatsApp, that requires approved **message templates** — a follow-up task.

7.3 Transactional email (SMTP) — ✅ wired, needs credentials

`lib/mailer.ts` + `lib/emails.ts`. Sends order confirmations, password reset, welcome, back-in-stock, abandoned-cart reminders. Dev-fallback logs to console. Set `SMTP_HOST/PORT/USER/PASS`, `EMAIL_FROM` (free options: Brevo, Resend, or Gmail App Password — examples in `.env.example`).

7.4 Courier shipping — ✅ Steadfast live-ready; Pathao/RedX mock

`lib/courier.ts`. **Steadfast** has a real API call (`portal.packzy.com`) that activates when `STEADFAST_API_KEY` / `STEADFAST_SECRET` are set. **Pathao** and **RedX** currently return mock tracking codes — the `createConsignment` shape is in place; add their real API calls (marked `TODO`) once merchant accounts exist. Triggered from the admin order page → "Ship with courier" → `POST /admin/orders/:id/ship`.

7.5 Social login — ✅ wired, needs credentials

`lib/oauth.ts` verifies Google (ID token) and Facebook (access token) server-side. Set `GOOGLE_CLIENT_ID`, `FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET` in backend, and `VITE_GOOGLE_CLIENT_ID` / `VITE_FACEBOOK_APP_ID` in `frontend/.env`. Buttons hide themselves until configured.

---

8. Environment variables

Copy `backend/.env.example` → `backend/.env` and `frontend/.env.example` → `frontend/.env`. Summary of backend vars:

Group	Vars
Core	`DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `PORT`, `CORS_ORIGIN`, `APP_URL`, `STORE_NAME`
Email	`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM`
WhatsApp	`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`, `WHATSAPP_ADMIN_TO`, `LOW_STOCK_THRESHOLD`
Courier	`STEADFAST_API_KEY`, `STEADFAST_SECRET`, `PATHAO_CLIENT_ID`, `PATHAO_CLIENT_SECRET`, `REDX_API_TOKEN`
Social	`GOOGLE_CLIENT_ID`, `FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`
Payment (to add)	`SSLC_STORE_ID`, `SSLC_STORE_PASSWORD`, `SSLC_IS_LIVE`

Frontend: `VITE_API_URL`, `VITE_GOOGLE_CLIENT_ID`, `VITE_FACEBOOK_APP_ID`.

---

9. Going to production

1. **Database → PostgreSQL.** In `schema.prisma` set `provider = "postgresql"`, point `DATABASE_URL` at a Postgres instance (Neon / Railway / Supabase, or `docker compose up -d` locally), then `npm run prisma:migrate` (switch from `db push` to real migrations for prod). The app code is DB-agnostic through Prisma.
2. **Secrets.** Generate a strong `JWT_SECRET`. Never commit `.env`.
3. **CORS.** `app.ts` currently allows any localhost origin for dev convenience — **tighten to the real storefront origin** (`config.corsOrigin`) before launch.
4. **File uploads.** Images save to local `backend/uploads/`. On ephemeral hosts (containers/serverless) this disk is not persistent — move to S3 / Cloudinary / a mounted volume.
5. **Hosting.** Frontend deploys cleanly to Vercel/Netlify (static build). Backend needs a persistent Node host (Railway, Render, a VPS, Fly.io) — not a purely static/serverless target, because of the background schedulers and file uploads.
6. **HTTPS + callback URLs.** SSLCommerz IPN/callback URLs and OAuth redirect URIs must be public HTTPS URLs registered in each provider's dashboard.

---

10. Suggested test plan (please verify end-to-end)

Storefront: register → browse → add to cart → apply a coupon → checkout (both inside & outside Dhaka shipping) → confirm the **server-computed total** matches → see it in order history → track it → cancel/return → leave a review with a photo → ask a product question.

Admin: log in as admin → dashboard stats → create/edit/delete a product → bulk import (CSV then PDF) → view an order → print invoice → ship with a courier → change order status → answer a Q&A question → run a sales report + CSV export → create a staff account with limited permissions and **verify it is blocked** from other sections → create a coupon and a promotion.

Integrations (once creds added): SSLCommerz sandbox payment incl. IPN validation · a real WhatsApp alert · a real order-confirmation email · a real Steadfast consignment · Google + Facebook login.

Regression watch-outs:

- All currency is integer BDT — no floats.
- The server must remain the source of truth for pricing; don't move totals to the client.
- `costPrice` must never appear in public API responses.
- Staff permission checks must stay server-side.

---

11. Known gaps / the roadmap after you

1. **SSLCommerz integration** — the one net-new build (§7.1). Highest priority.
2. **Product variants as real SKUs** — currently `colors` / `sizes` are display options on a single stock number. Making each variant its own SKU (own stock/price/image) was scoped and **deliberately deferred** because it rewires the pricing/stock/checkout path. Coordinate before starting; it deserves a focused pass with checkout testing.
3. **Pathao / RedX** real API calls (Steadfast is done).
4. **Customer-facing WhatsApp** order updates (needs approved message templates).
5. Move uploads off local disk for production.

---

12. Handy commands

```
# Backend
npm run dev            # start API (tsx watch)
npm run typecheck      # tsc --noEmit — must stay clean
npm run seed           # reload demo data
npm run prisma:push    # apply schema to DB (dev)
npm run prisma:migrate # create a migration (prod workflow)
npm run db:reset       # wipe + reseed

# Frontend
npm run dev            # Vite dev server
npm run build          # production build (must pass)
npm run lint           # eslint
```

Contact / repo: the code is in a private GitHub repository (`bow-and-tie`). Ask the owner for access. Start with the README, then this document, then `config.ts` and `routes/orders.ts` .

Everything except SSLCommerz runs today with mocks/console fallbacks; your work is to supply credentials, replace the couple of remaining mocks with real API calls, build the SSLCommerz flow, and test the whole thing end-to-end.